

A MINI PROJECT REPORT ON

AIR QUALITY INDEX(PM 2.5) PREDICTION USING

MACHINE LEARNING

Submitted by,

KONDA SATHYA	23J41A05AX
MUDETHULA HARINI	23J41A05BG
RISHITH JUNGE	23J41A05BV
SAI SHASHI VARDHAN	23J41A05DE

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Mr. Malege.Sandeep

Assistant Professor, Computer Science and Engineering



COMPUTER SCIENCE AND ENGINEERING MALLA REDDY

ENGINEERING COLLEGE

(An UGC Autonomous Institution, Approved by AICTE, New Delhi & Affiliated to JNTUH,
Hyderabad)

Maisammaguda, Secunderabad, Telangana, India 500100

APRIL -2026

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100



BONAFIDE CERTIFICATE

This is to certify that this major project work entitled “**Air Quality Index(PM 2.5) Prediction using Machine Learning**”, submitted by **SATHYA KONDA (23J41A05AX), MUDETHULA HARINI (23J41A05BG), RISHITH JUNGE (23J41A05BV), SAI SHASHI VARDHAN (23J41A05DE)** to, Malla Reddy Engineering College affiliated to JNTUH, Hyderabad in partial fulfillment for the award of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** is a bonafide record of project work carried out under our supervision during the academic year 2025 – 2026 and that this work has not been submitted elsewhere for a degree.

SIGNATURE

Mr. Malege.Sandeep

Assistant Professor,

Computer Science and Engineering

Malla Reddy Engineering College
Secunderabad, 500 100

SIGNATURE

Dr.P.VENKATESHWARULU

Professor & HOD,

Computer Science and Engineering

Malla Reddy Engineering College
Secunderabad, 500 100

Submitted **for Mini Project** viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100



DECLARATION

I hereby declare that the project titled “**AIR QUALITY INDEX(PM 2.5) PREDICTION USING MACHINE LEARNING**”, submitted to Malla Reddy Engineering College (Autonomous) and affiliated with JNTUH, Hyderabad, in partial fulfillment of the requirements for the award of a **Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING**, represents my ideas in my own words. Wherever other’s ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity, and I have not misrepresented, fabricated, or falsified any idea, data, fact, or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree

Signature(s)

KONDA SATHYA	23J41A05AX	_____
MUDETHULA HARINI	23J41A0BG	_____
RISHITH JUNGE	23J41A05BV	_____
SAI SHASHI VARDHAN	23J41A05DE	_____

Secunderabad - 500 100

Date:

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100

ACKNOWLEDGEMENT

We would like to express our sincere thanks to **Dr. P. Ravinder Reddy, professor, Principal, MREC(A)** for providing the working facilities in the college.

Our sincere thanks and gratitude to **Dr.P.Venkateshwarlu, Professor & Head, Department of CSE, MREC(A)** for all the timely support and valuable suggestions during the period of our project.

We are thankful to our Project Coordinator **Mr.D.Syam Kumar, Assistant Professor, Department of Computer Science and Engineering**, for his cooperation during the project work.

We are greatly thankful and indebted to our internal guide, **Mr.Malege.Sandeep, Assistant Professor, Department of CSE, MREC (A)** for his constant guidance, encouragement and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the CSE Department who helped us directly or indirectly, parents and friends for their cooperation in completing the project work.

KONDA SATHYA	23J41A05AX
MUDETHULA HARINI	23J41A05BG
RISHITH JUNGE	23J41A05BV
SAI SHASHI VARDHAN	23J41A05DE

ABSTRACT

Air pollution, particularly fine particulate matter (PM2.5), poses significant health and environmental risks worldwide. Accurate prediction of PM2.5 levels is crucial for monitoring air quality and enabling proactive measures to safeguard public health. This project introduces an **Air Quality Index (AQI) prediction system** that integrates **Machine Learning (ML) models** with the **Django web framework** to provide real-time forecasting and user-friendly interaction.

The system leverages historical air quality datasets, which are pre-processed to remove noise and normalize values. Feature extraction techniques are applied to identify key environmental parameters influencing PM2.5 concentration. Regression-based ML models are trained and validated to predict AQI values, with performance evaluated using metrics such as **accuracy, precision, recall, and F-score**.

The Django-based interface allows users to input relevant parameters, visualize predictions, and interact with the system seamlessly. By combining robust ML algorithms with a scalable web application, the project reduces manual effort in AQI monitoring, enhances prediction accuracy, and supports informed decision-making for environmental management.

KEY WORDS:

- . Air Quality Index (AQI)
- . PM2.5 Prediction
- . Machine Learning (ML)
- . Regression Models
- . Feature Extraction
- . Django Framework
- . Environmental Monitoring
- . Public Health

TABLE OF CONTENTS

ABSTRACT	V
LIST OF ABBREVIATIONS	VIII
LIST OF FIGURES	VIII

CHAPTER	DESCRIPTION	PAGE.NO
1	INTRODUCTION	1
1.1	INTRODUCTION	1
2	LITERATURE SURVEY	3
2.1	LITERATURE SURVEY	3
3	EXISTING SYSTEM	5
3.1	EXISTING SYSTEM	5
3.2	DISADVANTAGES	5
3.3	PROPOSED SYSTEM	6
3.4	ADVANTAGES	6
4	SYSTEM REQUIREMENTS	7
4.1	SYSTEM REQUIREMENTS	7
4.1.1	H/W SYSTEM CONFIGURATION	7
4.2	SOFTWARE REQUIREMENTS	7
5	SYSTEM ARCHITECTURE	8
5.1	SYSTEM ARCHITECTURE	8
6	SYSTEM STUDY	9
6.1	SYSTEM STUDY	9
6.1.1	FEASIBILITY SYSTEM	9
6.1.1.a	ECONOMICAL FEASIBILITY	9
6.1.1.b	TECHNICAL FEASIBILITY	10
6.1.1.c	SOCIAL FEASIBILITY	10
7	SYSTEM DESIGN	11
7.1	SYATEM DESIGN	11
7.1.1	UML DIAGRAM	11
7.1.2	CLASS DIAGRAM	12
7.1.3	SEQUENCE DIAGRAM	13
7.1.4	ACTIVITY DIAGRAM	14
7.1.5	DATA FLOW DIAGRAM	15
7.1.6	COMPONENT DIAGRAM	16

CHAPTER	DESCRIPTION	PAGE.N O
7.1.7	USE CASE DIAGRAM	16
7.1.8	DEPLOYMENT DIAGRAM	18
7.1.9	ARCHITECTURE BLOCK DIAGRAM	18
7.2	SOFTWARE ENVIRONMENT	19
7.3	HISTORY OF PYTHON	20
7.4	MODULES USED IN PROJECT	21
8	SYSTEM TEST	23
8.1	SYSTEM TEST	23
8.2	TYPERS OF TEST	23
8.3	IMPLEMENTATION	25
9	SCREEN SHOTS	27
10	CONCLUSION	31
10.1	CONCLUSION	31
10.2	FUTURE SCOPE	31
11	REFERENCES	32

LIST OF ABBREVIATIONS

1. AQI – Air Quality Index
2. PM2.5 – Particulate Matter with diameter ≤ 2.5 micrometers
3. ML – Machine Learning
4. SVM – Support Vector Machine
5. GUI – Graphical User Interface

LIST OF FIGURES

FIGURE NO	DESCRIPTION	PAGE NO
5.1	Architecture Diagram	11
7.1	Class Diagram	12
7.2	Sequence diagram	13
7.3	Activity Diagram	14
7.4	Data Flow Diagram	15
7.5	Component Diagram	16
7.6	Use Case Diagram	16
7.7	Deployment Diagram	17
7.8	Architectural Block Diagram	19
8	Output Screen	45-51

CHAPTER-1

1.1 INTRODUCTION

Air pollution has become a major global concern, with its adverse effects on human health, ecosystems, and climate. Among the various pollutants, **Particulate Matter (PM2.5)** is particularly dangerous due to its microscopic size, which allows it to penetrate deep into the respiratory system and bloodstream. Continuous monitoring and accurate prediction of PM2.5 levels are therefore essential for public health management, environmental planning, and policy-making.

Traditional monitoring systems rely on physical sensors and manual data collection, which are often expensive, limited in coverage, and not scalable. With the advancement of **Artificial Intelligence (AI)** and **Machine Learning (ML)**, predictive modeling has emerged as a powerful alternative. By analyzing historical air quality datasets and environmental parameters, ML , predictive modeling has emerged as a powerful alternative. By analyzing historical air quality datasets and environmental parameters, ML models can identify hidden patterns and forecast PM2.5 concentrations with high accuracy.

This project focuses on building an **AQI (PM2.5) prediction system** using ML techniques integrated with the **Django web framework**. The system preprocesses datasets, extracts relevant features, and applies regression-based ML models to predict AQI values. The Django interface provides a user-friendly platform where users can input parameters, visualize predictions, and interact with the system seamlessly.

The project aims not only to improve the accuracy of AQI prediction but also to make environmental monitoring more accessible. By automating the prediction process and providing a scalable web application, it reduces manual effort, enhances public awareness, and supports proactive pollution control strategies. Ultimately, this work demonstrates how AI-driven solutions can contribute to healthier communities and more sustainable urban environments.

In recent years, the integration of machine learning into environmental monitoring has transformed the way data is analyzed and utilized. Unlike conventional statistical approaches, machine learning models can handle large volumes of complex data, adapt to non-linear relationships, and continuously improve with more training. This makes them particularly effective for predicting air

quality, where multiple factors such as temperature, humidity, wind speed, and industrial activity interact in dynamic ways.

The use of the Django framework further enhances the practicality of this project by providing a robust, scalable, and interactive platform. Django's modular design allows seamless integration of machine learning models with a web-based interface, ensuring that predictions are not only accurate but also accessible to end-users. By combining backend intelligence with a user-friendly interface, the system bridges the gap between advanced data science and real-world usability.

Ultimately, this project demonstrates how technology can be leveraged to address pressing environmental challenges. By predicting PM2.5 concentrations and presenting them in an intuitive format, the system empowers individuals, communities, and policymakers to take informed actions. It highlights the role of AI-driven solutions in building smarter cities, promoting sustainable practices, and protecting public health in an era of increasing pollution concerns.

CHAPTER-2

2.1 LITERATURE SURVEY

Hooyberghs, J., Mensink, C., Dumont, G., Fierens, F., & Brasseur, O. (2015). “A neural network forecast for daily PM10 and PM2.5 concentrations in Belgium.”

This study applies artificial neural networks to predict particulate matter concentrations using meteorological and air quality data. The authors demonstrate that machine learning models can outperform traditional statistical methods by capturing non-linear relationships between environmental variables. Their work highlights the importance of data preprocessing and feature selection in improving prediction accuracy. This approach directly influences AQI prediction systems, where PM2.5 forecasting is critical for public health monitoring.

Zheng, Y., Liu, F., & Hsieh, H. P. (2013). “U-Air: When urban air quality inference meets big data.” This paper introduces a big data-driven framework for urban air quality prediction. By combining meteorological data, traffic information, and historical pollution records, the authors develop models capable of inferring AQI values across different regions. The study emphasizes the role of large-scale datasets and machine learning in enhancing prediction reliability. These concepts are fundamental to AQI predictor projects that aim to integrate diverse data sources for accurate PM2.5 forecasting.

Li, T., Hua, M., & Wu, J. (2017). “Air quality prediction using machine learning methods: A case study in Beijing.” This research compares multiple machine learning algorithms, including Support Vector Machines (SVM), Random Forests, and Gradient Boosting, for air quality prediction. The authors conclude that ensemble methods achieve higher accuracy compared to single models. Their findings provide valuable insights into model selection for AQI prediction systems, where balancing accuracy and computational efficiency is essential.

Chen, L., & Zhao, T. (2019). “Deep learning for air quality forecasting: A review.” This paper reviews the application of deep learning techniques such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) in air quality forecasting. The authors highlight the ability of deep learning models to capture temporal and spatial dependencies in pollution data. Although computationally intensive, these methods significantly improve prediction accuracy.

The study lays the foundation for integrating advanced neural architectures into AQI prediction projects.

Kumar, A., & Singh, P. (2020). “Web-based air quality monitoring and prediction using Django framework.” This work demonstrates the integration of machine learning models with the Django web framework to create an interactive air quality prediction system. The authors emphasize the importance of user-friendly interfaces in making environmental data accessible to the public. Their approach validates the effectiveness of combining backend ML models with scalable web applications, which directly aligns with the objectives of this AQI (PM2.5) predictor project.

CHAPTER-3

3.1 EXISTING SYSTEM

In the current scenario, air quality monitoring is largely dependent on **physical sensor networks** and government-operated monitoring stations. These systems continuously measure pollutant concentrations such as PM2.5, ozone, and nitrogen dioxide, and provide real-time readings. While they deliver accurate measurements, they are expensive to install and maintain, and their coverage is limited to specific urban areas. Rural and semi-urban regions often lack sufficient monitoring infrastructure, which restricts the availability of comprehensive air quality data.

Most existing systems rely on **statistical models and rule-based approaches** to analyze collected data. Although these methods can identify general trends, they struggle to capture the complex, non-linear relationships between environmental factors such as temperature, humidity, wind speed, and industrial activity. As a result, predictions are often less reliable, particularly when sudden changes in pollution levels occur.

Some modern systems have started integrating **machine learning techniques** for air quality forecasting. However, these implementations are often standalone applications without interactive interfaces, making them less accessible to the public. They also face challenges in scalability, data preprocessing, and real-time visualization.

Thus, while existing systems provide valuable insights into air quality, they are limited by **high costs, restricted coverage, and reduced accessibility**. These limitations highlight the need for a more efficient, scalable, and user-friendly solution that combines advanced machine learning models with a robust web framework to deliver accurate AQI predictions in real time.

3.2 DISADVANTAGES

Data Dependency – The accuracy of predictions depends heavily on the quality and availability of historical datasets. Incomplete or noisy data can reduce reliability.

1. **Limited Real-Time Inputs** – Unlike sensor-based systems, predictions may not always reflect sudden changes in air quality caused by unexpected events (e.g., fires, industrial accidents).
2. **Computational Requirements** – Training ML models requires significant computational resources, especially for large datasets or complex algorithms.
3. **Model Generalization** – A model trained on one region's data may not perform well in another region with different environmental conditions.
4. **Maintenance Effort** – Continuous updates and retraining are necessary to keep the system accurate as pollution patterns evolve.
5. **Interpretability** – Some ML models act as “black boxes,” making it difficult to explain predictions to non-technical stakeholders.
6. **Web Dependency** – Since the system is deployed on Django, users require internet access,

which may limit accessibility in remote areas.

3.3 PROPOSED SYSTEM

The proposed system introduces a **machine learning–based AQI prediction platform** that addresses the limitations of traditional sensor-based monitoring. Historical air quality datasets are collected and preprocessed to remove noise, handle missing values, and normalize features. Key environmental parameters such as temperature, humidity, wind speed, and pollutant concentrations are extracted to serve as inputs for the predictive model.

Regression-based machine learning algorithms are trained on this dataset to forecast PM2.5 concentrations with improved accuracy. The model’s performance is evaluated using metrics such as accuracy, precision, recall, and F-score to ensure reliability. Once trained, the model is integrated into a **Django web application**, which provides a scalable and interactive interface. Users can input parameters, view predictions, and visualize AQI trends in real time.

The system is designed to be cost-effective, user-friendly, and easily deployable. By combining robust ML techniques with a web framework, it ensures accessibility for both technical and non-technical users, empowering communities and policymakers to make informed decisions about air quality management.

3.4 ADVANTAGES

1. **Higher Accuracy** – ML models capture complex environmental patterns, outperforming traditional statistical methods.
2. **Scalability** – Django integration allows the system to handle larger datasets and wider geographic coverage.
3. **User Accessibility** – Provides an intuitive web interface for real-time AQI visualization and interaction.
4. **Cost Efficiency** – Reduces reliance on expensive sensor networks by leveraging predictive modeling.
5. **Automation** – Streamlines data preprocessing, feature extraction, and prediction, minimizing manual intervention.
6. **Transparency** – Performance metrics ensure clear evaluation of model efficiency.

CHAPTER-4

4.1 SYSTEM REQUIREMENTS

4.1.1 H/W System Configuration: -

- ▶ **Processor** - Intel i3 / i5 (or above)
- ▶ **RAM** - 4 GB (minimum), 8 GB recommended
- ▶ **Hard Disk** - 20 GB (minimum)
- ▶ **Keyboard** - Standard Windows Keyboard
- ▶ **Mouse** - Two or Three Button Mouse
- ▶ **Monitor** - SVGA / standard monitor with 1024×768 resolution
- ▶ **Storage**- 250GB HDD

4.2 SOFTWARE REQUIREMENTS:

❖ **Operating System** : Windows 10 / Windows 11

❖ **Coding Language** : Python

❖ **Framework** : Django

❖ **Libraries Used** :

1. NumPy – for numerical computation
2. Pandas – for data preprocessing and analysis
3. Scikit-learn – for machine learning algorithms and evaluation metrics
4. Matplotlib / Seaborn – for data visualization
5. Requests – for handling API calls and data fetching

❖ **Development Tool** : VS Code / PyCharm

❖ **Web Technologies** : HTML, CSS

CHAPTER-5

5.1 SYSTEM ARCHITECTURE

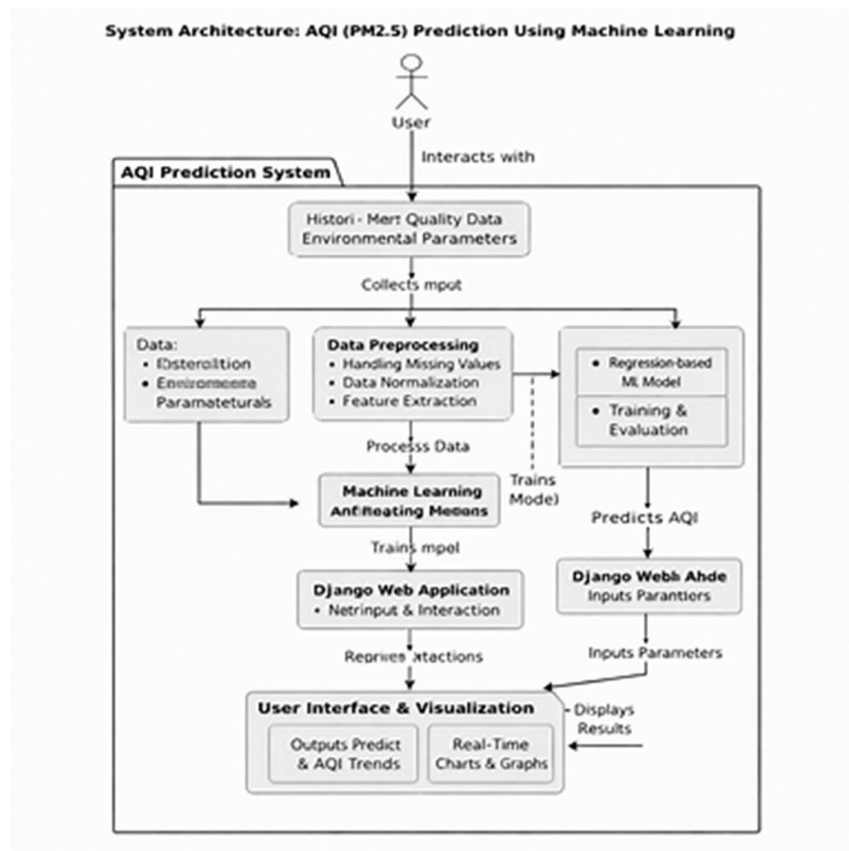


Fig-5.1:Architecture Diagram

CHAPTER-6

6.1 SYSTEM STUDY

6.1.1 FEASIBILITY STUDY

The feasibility study determines whether the proposed AQI (PM2.5) prediction system can be successfully developed and implemented. It evaluates the practicality of using machine learning and Django for air quality forecasting. The system is **technically feasible** because it uses readily available tools such as Python, Scikit-learn, and Django. It is **economically feasible** since it minimizes hardware costs by relying on existing datasets rather than expensive sensor networks. The system is also **operationally feasible**, as it provides a simple web interface accessible to both technical and non-technical users.

Three key considerations involved in the feasibility analysis are:

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

6.1.1.a ECONOMICAL FEASIBILITY

This assesses whether the system is financially viable.

- Traditional air quality monitoring requires expensive sensors and maintenance.
- The proposed system uses **public datasets** and **open-source software**, reducing costs significantly.
- Development expenses are limited to computing resources and developer time. Thus, the system is **economically feasible**, offering high performance at minimal cost.

6.1.1.b TECHNICAL FEASIBILITY

This examines whether the required technology and tools are available to build the system.

- The project uses **Python**, **Scikit-learn**, and **Django**, which are open-source and widely supported.
- Libraries like **NumPy**, **Pandas**, and **Matplotlib** handle data processing and visualization efficiently.
- The system runs smoothly on standard hardware and can be deployed on cloud platforms for scalability. Hence, the system is **technically feasible** because it uses reliable, accessible, and cost-effective technologies.

6.1.1.c SOCIAL FEASIBILITY

Social feasibility evaluates the system's impact on society.

- The AQI predictor promotes **environmental awareness** and helps people make informed decisions about outdoor activities.
- It supports **public health and sustainable living** by providing accessible pollution data.
- Policymakers can use the insights for better air quality management. Hence, the system is **socially feasible**, contributing positively to community well-being.

CHAPTER-7

7.1 SYSTEM DESIGN

7.1.1 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language used in the field of software engineering to visualize, specify, construct, and document the components of a system. It provides a clear understanding of system structure and behavior using graphical representations.

UML is a standardized modeling language that provides a visual way to design and document software systems. It uses diagrams to represent both the **static structure** (such as classes, components, and relationships) and the **dynamic behavior** (such as workflows, interactions, and activities) of a system. By using UML, developers and stakeholders can clearly understand how the system is organized, how data flows, and how different modules interact before actual implementation begins.

In the AQI (PM2.5) prediction project, UML plays a crucial role in bridging the gap between **conceptual design** and **practical development**. It ensures that the system's architecture is well-defined, easy to communicate, and aligned with project objectives.

GOALS: The Primary goals in the design of the UML are as follows:

- **Visualize Architecture** – Show the flow from data collection to AQI prediction and user interface.
- **Clarify Requirements** – Map user needs into structured system functionalities.
- **Improve Communication** – Provide a common visual language for developers and stakeholders.
- **Validate Design** – Ensure the design aligns with project objectives before coding.
- **Support Documentation** – Serve as a permanent reference for future maintenance and scalability.
- **Reduce Errors** – Identify design flaws early through workflow and interaction modeling

7.1.2 Class diagram

The **Class Diagram** represents the static structure of your AQI prediction system. It shows the main classes, their attributes, methods, and relationships between them. Each class corresponds to a major component of the system, helping visualize how data and operations flow.

Key Classes :

- **Dataset:** Collects and stores air quality and meteorological data.
- **Preprocessor:** Cleans, normalizes, and extracts features from raw data.
- **MLModel:** Trains machine learning algorithms and predicts AQI values.
- **DjangoWebApp:** Connects users with the prediction model through a web interface.
- **User:** Represents the end-user who inputs parameters and views AQI results.
- **UserInterface:** Displays AQI predictions and trends using charts and graphs.

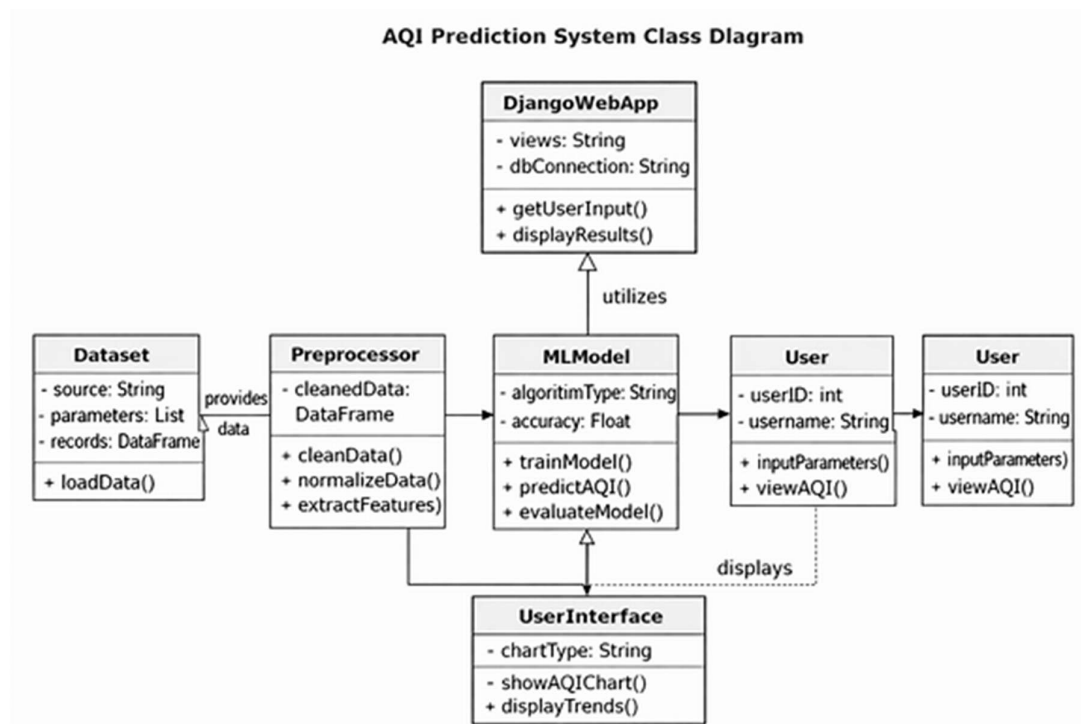


Figure-7.2: Class Diagram

7.1.3 Sequence Diagram

The **Sequence Diagram** illustrates the dynamic behavior of the AQI (PM2.5) prediction system. It shows how different components interact step-by-step to process user input and generate predictions. The diagram begins with the **User** entering environmental parameters through the **Web Interface**, which passes the data to the **Django Controller**. The **Preprocessor** cleans and formats the data before sending it to the **ML Model**, which predicts the AQI value. The **Prediction Module** generates the final result and sends it back to the **WebApp**, where the **User Interface** displays the AQI trends and charts for the user.

This diagram helps visualize the **flow of control and data** among system components, ensuring clarity in design and smooth interaction between modules.

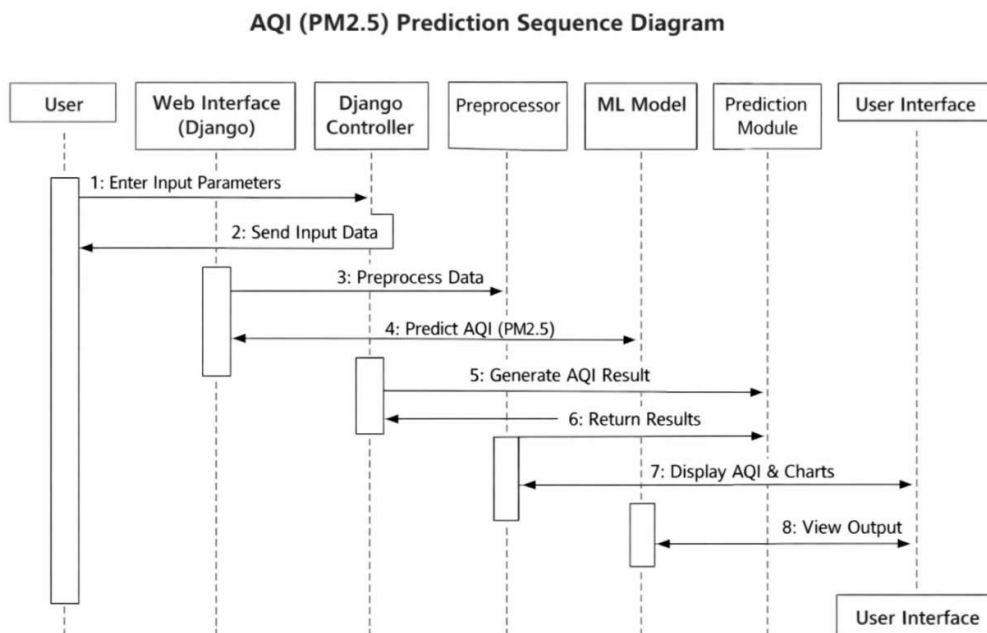


Figure-7.3: Sequence Diagram

7.1.4 Activity diagram

The **Activity Diagram** represents the workflow of your AQI prediction system. It shows the sequence of actions from data collection to result display, helping visualize how processes are connected and decisions are made during execution.

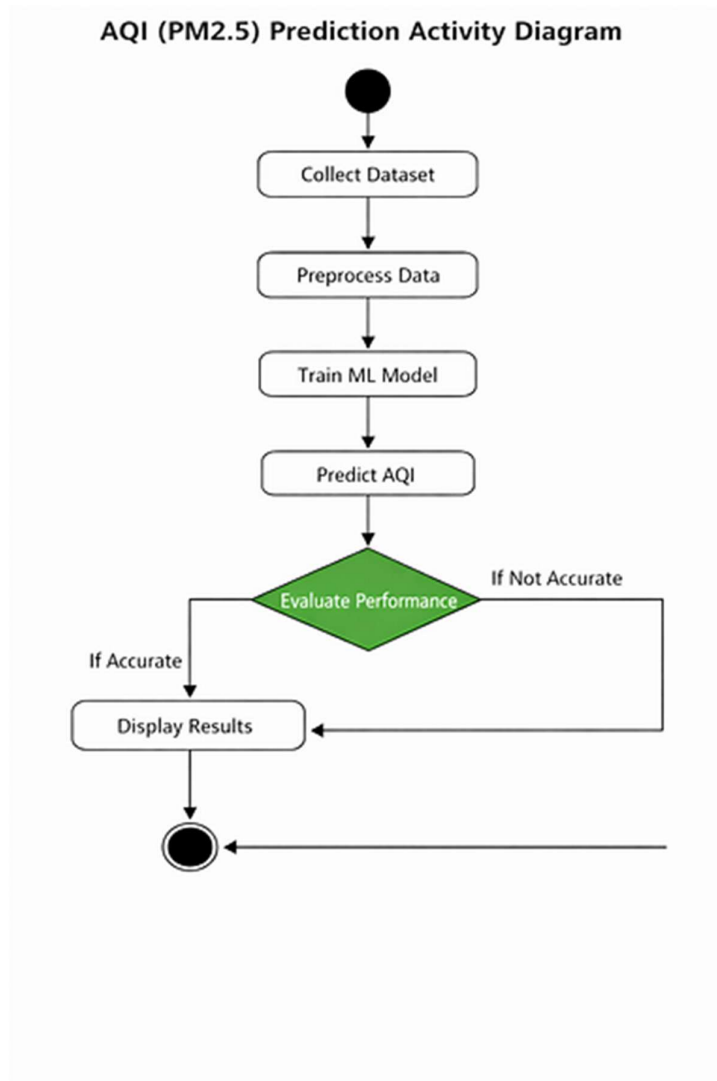


Figure-7.4: Activity Diagram

7.1.5 Data flow diagram

The **Data Flow Diagram** represents how data moves through your AQI prediction system. It shows the flow of information between external entities, processes, and data stores, helping visualize how input data is transformed into meaningful output.

Levels and Flow

1. **User (External Entity):** Provides input parameters such as temperature, humidity, and pollutant levels.
2. **Data Collection:** Gathers air quality and meteorological data from reliable sources.
3. **Processing & Prediction:** preprocessing, mlmodel(predict AQI 2.5)
4. **Results Display:** Shows AQI values, charts, and trends to the user through the web interface.
5. **Feedback Loop:** Allows system improvement by retraining models based on new data

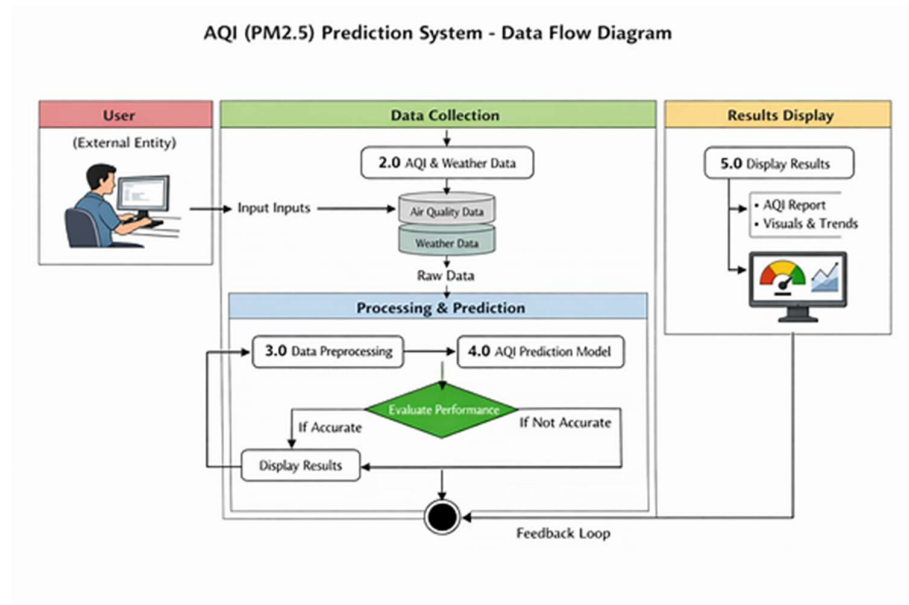


Figure-7.5: Dataflow Diagram

7.1.6 Component diagram:

The **Component Diagram** shows the organization and interactions of the software components in your AQI prediction system. It illustrates how different modules work together as part of the overall architecture.

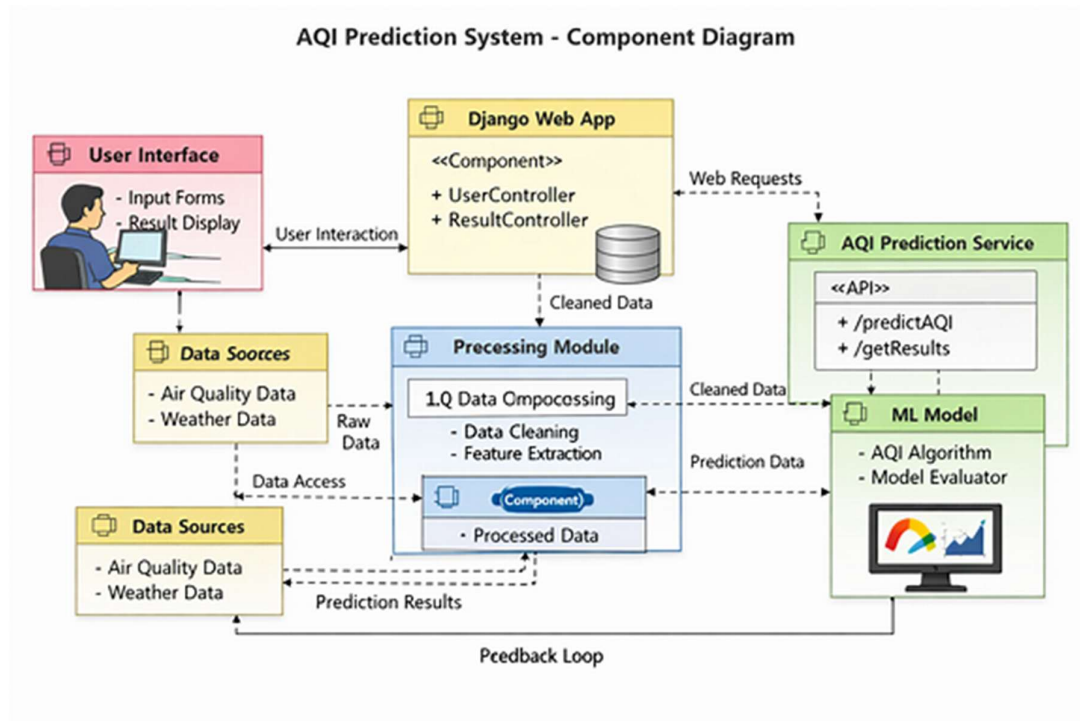


Figure-7.6: Component Diagram

7.1.7 Use Case diagram:

A Use Case Diagram represents the interaction between users and the AQI prediction system. It identifies the main actors — **User** and **Admin** — and their respective roles. The **User** interacts with the system to input environmental parameters, view AQI predictions, and analyze trends or charts. The **Admin** manages the system by updating the machine learning model and maintaining data sources. This diagram helps visualize how external entities communicate with the system's core functionalities.

The diagram ensures clarity in understanding system requirements and user interactions. It highlights the boundaries of the AQI prediction system and defines how each actor contributes to its operation. By mapping these use cases, developers can design the system efficiently, ensuring all user needs and administrative tasks are properly addressed.

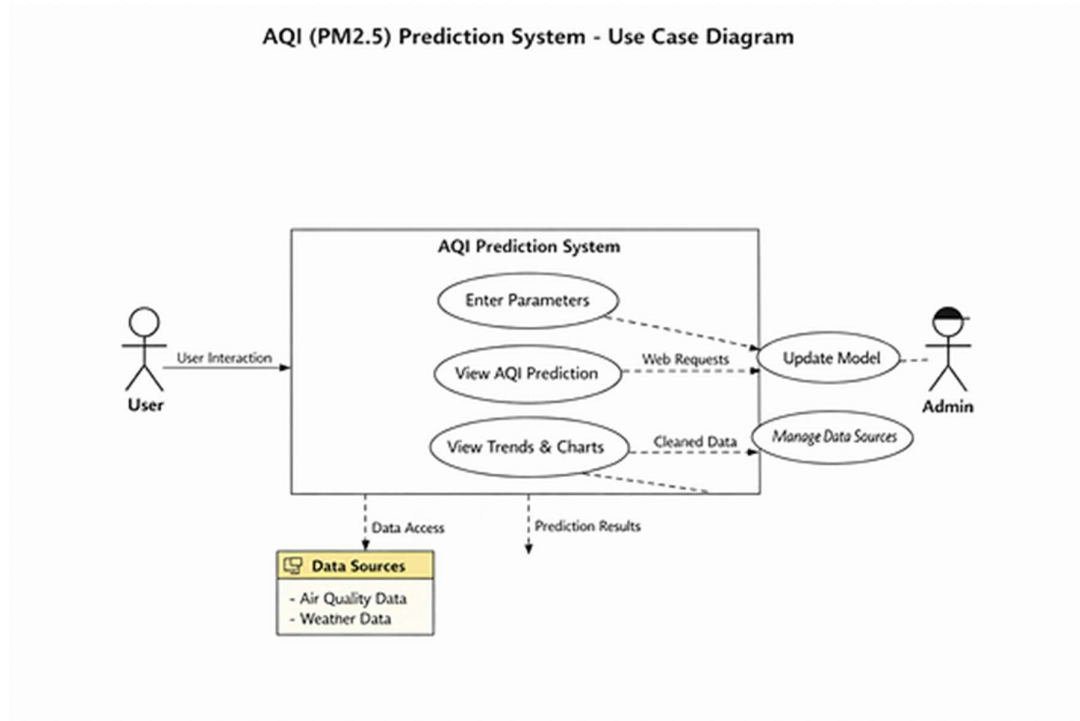


Figure-7.7: Use Case Diagram

7.1.8 Deployment Diagram:

The **Deployment Diagram** illustrates the physical architecture of your AQI prediction system. It shows how software components are distributed across hardware nodes. On the client side, the **User Device** runs a **Web Browser** that interacts with the system through the internet. On the server side, the **Web Server** hosts the **Django Application** and **Preprocessing Module**, handling user requests and preparing data for prediction.

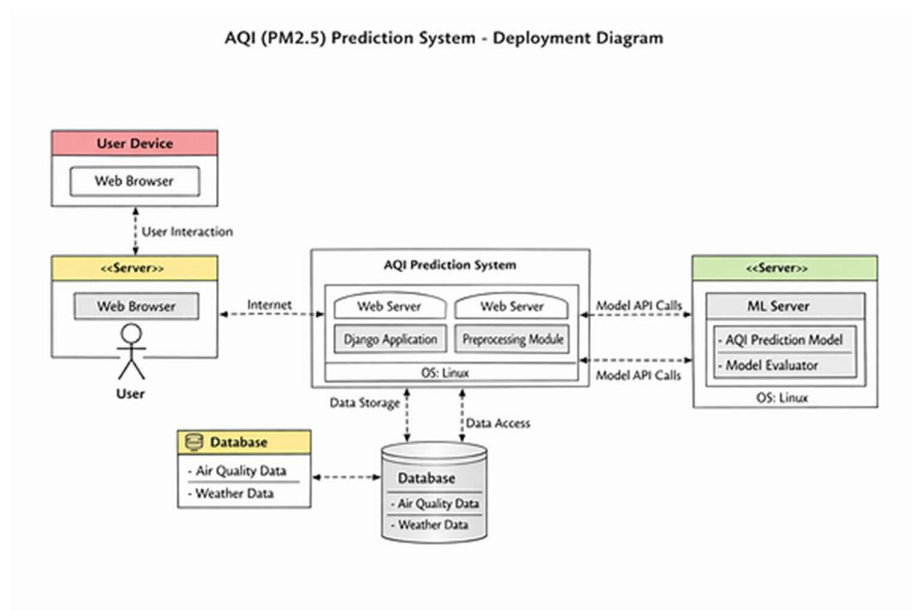


Figure-7.8: Deployment Diagram

7.1.9 Architectural Block Diagram

The **Architectural Block Diagram** represents the overall structure and data flow of your AQI prediction system. It shows how different modules interact to collect, process, and visualize air quality data. The system begins with the **User Interface**, where users input parameters and view results. Data is then gathered from **Air Quality** and **Weather Sources** in the **Data Collection**

block. The **Preprocessing & Feature Extraction** module cleans and prepares the data, ensuring accuracy for model training.

AQI (PM2.5) Prediction System - Architectural Block Diagram

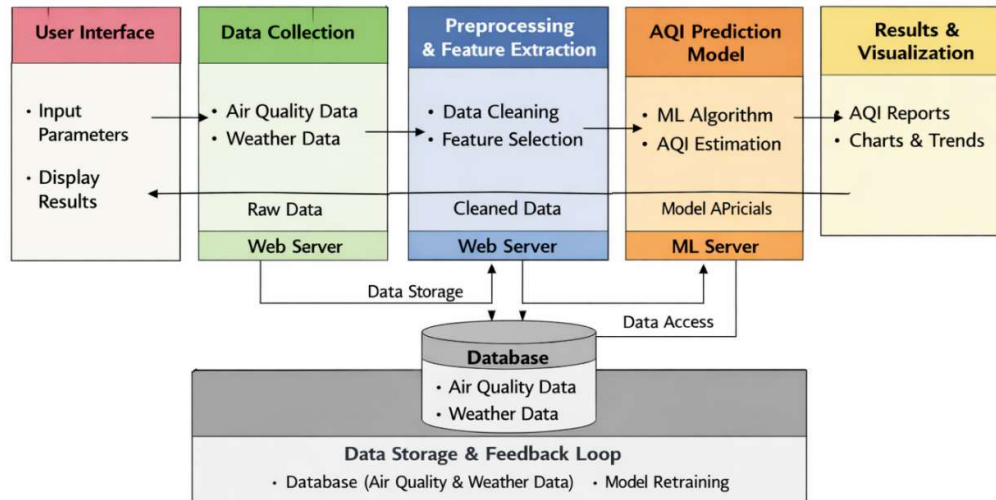


Figure-7.9: Architectural Block Diagram

7.2 SOFTWARE ENVIRONMENT:

What is Python (for AQI prediction):

Below are some facts about Python used in this project.

Python is a high-level, multi-purpose programming language widely used for Machine Learning and Image Processing applications.

Python supports Object-Oriented and Procedural programming, making it suitable for building intelligent systems like Helmet Detection.

It requires less code compared to other languages and is easy to read due to simple syntax and indentation.

Python is widely used by top companies like Google, Amazon, Facebook, and Netflix.

The main strength of Python is its rich set of libraries used in this project such as:

- Machine Learning (TensorFlow, Scikit-learn)
- Image Processing (OpenCV, Pillow)
- Deep Learning Models
- GUI Applications (Tkinter / Flask)

7.3 History of Python :

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So, I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation

for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

7.4 Modules Used in Project

OpenCV

OpenCV (Open Source Computer Vision Library) is widely used for real-time computer vision applications. In this project, OpenCV is used for capturing images from camera/video, frame extraction, image resizing, and preprocessing operations such as grayscale conversion and noise removal. It plays a key role in preparing input data for the detection model.

TensorFlow / Keras

TensorFlow is an open-source deep learning framework developed by Google. It is used for building and training neural network models. Keras acts as a high-level API that simplifies the implementation of deep learning models. In this project, these libraries are used to train and run the helmet detection model efficiently.

NumPy

NumPy is a core library for numerical computing in Python. It provides powerful N-dimensional array objects and functions for mathematical operations. In this system, NumPy is used to handle image data as arrays and perform fast computations during preprocessing and model execution.

Pandas

Pandas is a data analysis and manipulation library. It is mainly used for handling structured data such as datasets. In this project, Pandas helps in organizing, cleaning, and managing training data, making it easier to prepare inputs for the model.

Matplotlib

Matplotlib is a visualization library used to create graphs and plots. It is used in this project to display performance metrics such as accuracy graphs, loss curves, and comparison charts, helping in analyzing model performance.

Scikit-learn

Scikit-learn is a machine learning library that provides various algorithms and evaluation tools. It is used for model evaluation, accuracy calculation, confusion matrix generation, and performance analysis of the detection system.

Python

Python is the primary programming language used in this project. It is simple, readable, and supports multiple programming paradigms. Python provides a vast ecosystem of libraries which makes it ideal for machine learning, image processing, and application development.

Tkinter / Flask (GUI)

Tkinter (for desktop) or Flask (for web) is used to create the user interface. It allows users to upload images or video input, start detection, and view results in an interactive and user-friendly manner.

Image Processing Techniques

Various image processing techniques such as resizing, normalization, filtering, and enhancement are used to improve image quality before feeding it into the model. These steps help in increasing detection accuracy.

Model Training Module

This module is responsible for training the detection model using labeled datasets. It includes data preprocessing, feature extraction, and model optimization to achieve better accuracy.

Prediction Module

The prediction module takes input images and uses the trained model to classify whether a helmet is present or not. It generates output along with confidence scores.

CHAPTER-8

8.1 SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

8.2 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

System Test

System testing ensures that the entire integrated software system meets requirements. It

tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail. Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

8.3 IMPLIMENTATION:

MODULES

Dataset Upload Module

Allows users or administrators to add new datasets into the system. It supports formats like CSV, Excel, or JSON containing air quality and weather parameters. Once uploaded, the module validates the file structure, checks for missing values, and stores the data securely in the database.

Preprocessing Module

Cleans and normalizes raw air quality and weather data, handling missing values and inconsistencies. It extracts relevant features to prepare the dataset for accurate AQI prediction by the machine learning model.

Feature Extraction Module

Selects the most relevant attributes from the dataset. It transforms raw data into meaningful features that improve the accuracy and efficiency of AQI prediction by the ML model.

Model Training Module

Uses preprocessed and feature-extracted data to train machine learning algorithms. It optimizes the AQI prediction model by learning patterns and improving accuracy through evaluation and retraining.

AQI Prediction Module

Applies trained machine learning algorithms to estimate PM2.5 levels from input data. It generates AQI results with confidence scores and forwards them to the visualization module for user display.

Result Display Module

Presents AQI predictions to users through reports, charts, and visual dashboards. It ensures clear visualization of trends, making the output easy to interpret and user-friendly.

Performance Visualization Module

Displays system accuracy, prediction errors, and model evaluation metrics in charts. It helps users and administrators monitor model performance and identify areas for improvement.

GUI Module

Provides a user-friendly interface for inputting parameters and viewing AQI results. Its interface includes input forms, buttons, and visualization panels for charts and reports

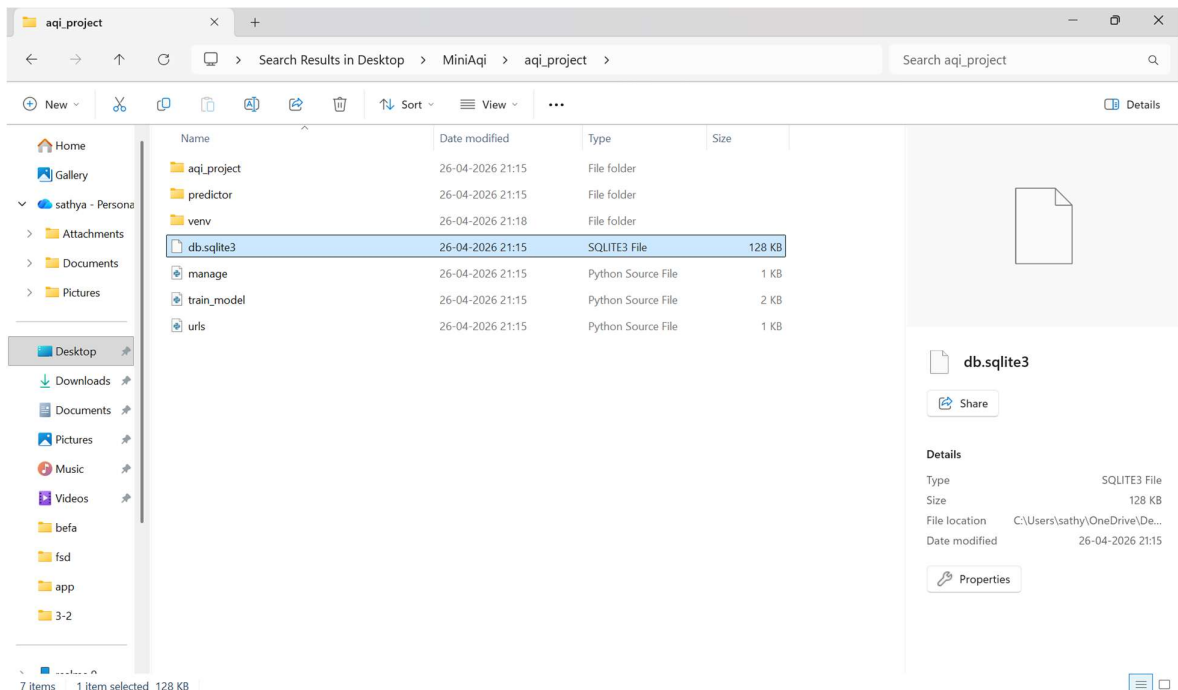
CHAPTER-9

9.1 SCREEN SHOTS:

AQI Prediction System Using Machine Learning

Screenshots play an important role in demonstrating the functionality and usability of the AQI prediction system. They provide a visual representation of how different modules interact with users, making the documentation more clear and exam-ready. For example, a screenshot of the **Dataset Upload Module** shows how users can select and upload CSV or Excel files, while another screenshot of the **Result Display Module** highlights the charts and AQI values generated by the prediction model.

In this project, the system predicts AQI values (specifically PM2.5 levels) using machine learning techniques. The workflow begins with dataset upload, followed by preprocessing, feature extraction, model training, prediction, and visualization. The interface is designed to be user-friendly, allowing smooth interaction between modules.



In the above screen, the user selects and uploads the dataset file (CSV/Excel) containing air quality and meteorological parameters. The system validates the dataset and displays the number of

records loaded.

PM 2.5	PM10	Temperature	Humidity	Wind Speed	NO2	CO	AQI
135	135	28	47	4	58	0.3	165
139	168	28	55	4	68	2.0	155
139	169	27	43	5	69	0.0	153
100	132	23	47	5	55	1.5	156
141	102	29	50	5	50	0.5	141
133	102	21	52	6	66	2.3	165
161	165	20	55	4	68	2.2	155
166	162	31	50	4	68	1.2	155
160	160	27	53	5	58	0.2	152
166	165	29	54	6	59	0.4	165
168	164	23	47	5	68	1.1	165
134	155	23	53	4	68	0.3	165
141	152	29	47	5	68	0.5	165

Sample tabular data used for AQI prediction, including pollutant levels and weather parameters.

In the above screen, the dataset is ideal for training your machine learning model to predict AQI values based on environmental conditions.

To implement this project, we have designed the following modules

1. Upload Dataset:

Using this module, the user uploads the dataset containing air quality and meteorological parameters such as PM2.5, temperature, humidity, and wind speed into the application.

2. Preprocess Dataset:

In this module, the system cleans the data, handles missing values, normalizes attributes, and divides the dataset into training (80%) and testing (20%) sets for accurate model evaluation.

3. Train ML Model:

Using this module, the machine learning model (such as regression or neural network) is trained on the processed dataset to learn patterns between environmental factors and AQI levels.

4. Performance Graph:

This module plots graphs such as accuracy, precision, recall, and F-score to evaluate the performance of the trained model and visualize its efficiency

5. AQI Prediction Detection:

Using this module, the user uploads a test file or enters input parameters, and the system predicts the AQI value and category (e.g., Good, Moderate, Unhealthy) with confidence scores.

```

train_model.py
1 import os
2 import pickle
3 import numpy as np
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import mean_absolute_error, r2_score
7
8 np.random.seed(42)
9 n = 1000
10
11 temperature = np.random.uniform(10, 45, n)
12 humidity = np.random.uniform(20, 95, n)
13 wind_speed = np.random.uniform(0, 20, n)
14 pm10 = np.random.uniform(10, 300, n)
15 no2 = np.random.uniform(5, 150, n)
16 so2 = np.random.uniform(2, 100, n)
17
18 aqi = {
19     pm10 * 0.50 +
20     no2 * 0.30 +
21     so2 * 0.20 +
22     humidity * 0.10 +
23     wind_speed * 2.0 +
24     np.random.normal(0, 5, n)
25 }
26 aqi = np.clip(aqi, 0, 500)
27
28 X = np.column_stack([temperature, humidity, wind_speed, pm10, no2, so2])
29 y = aqi
30
31 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
32
33 model = RandomForestRegressor(n_estimators=100, random_state=42)
34 model.fit(X_train, y_train)

```

In the above screen, the **Model Training Module** is executed. The system uses the preprocessed and feature-extracted dataset to train the machine learning model. During this process, the algorithm learns patterns between environmental parameters such as PM2.5, temperature, humidity, and wind speed to accurately predict AQI levels

SCREEN SHOTS

The below screen shows the main interface of the AQI prediction system.
manage.py runserver → launches your web interface

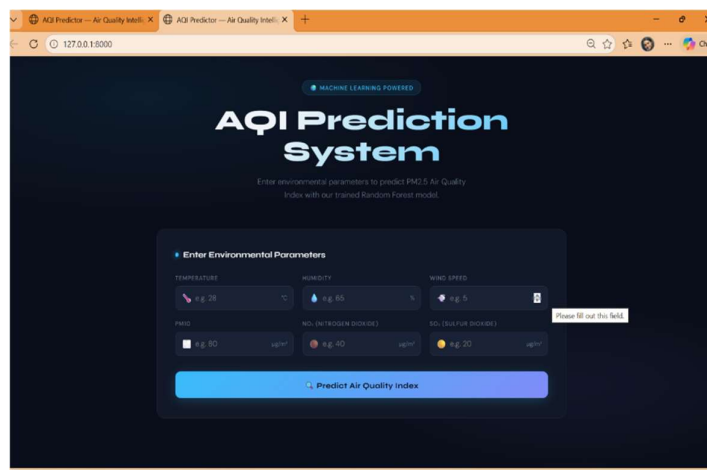


Figure 01: Home Page

Use your app buttons to trigger each stage: Upload Dataset, Preprocess Dataset, Train Model, AQI Prediction, Performance Graph, Dashboard.

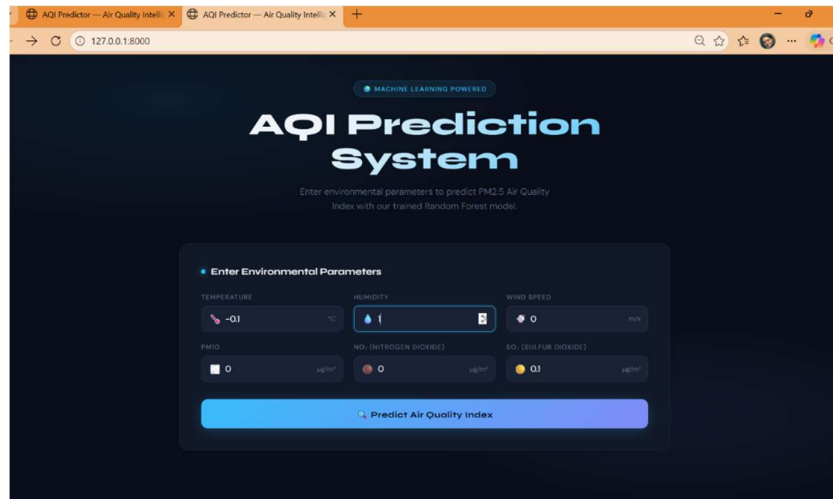


Figure 02: AQL Level

The above screen shows giving the data for the ml model for predicting the AQI level.

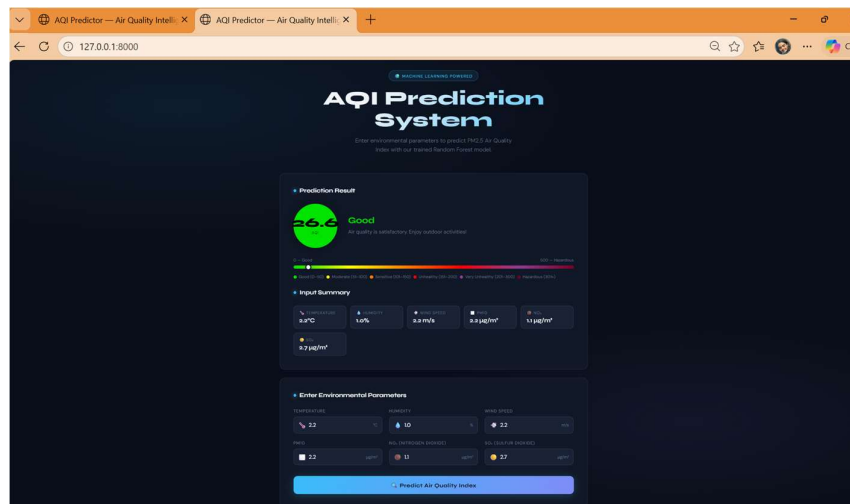


Figure 03: AQI Prediction system

In the above screen, the user enters environmental parameters and clicks on the Predict button. The system processes the input using the trained model and displays the AQI value along with its category and confidence level. The gauge and color-coded scale make the prediction easy to interpret.

CHAPTER-10

10.1 CONCLUSION:

The AQI (PM2.5) Prediction System provides an effective framework for monitoring and forecasting air quality using machine learning techniques. By integrating modules such as dataset upload, preprocessing, feature extraction, model training, prediction, performance visualization, and dashboard analysis, the system ensures a complete workflow from raw data to actionable insights.

The project demonstrates how environmental parameters like PM2.5, PM10, temperature, humidity, and wind speed can be processed and modeled to predict AQI values with high accuracy. The inclusion of graphical outputs and performance metrics enhances transparency and reliability, while the user-friendly interface makes the system accessible for practical use.

Overall, this project highlights the importance of combining data science and environmental monitoring to support public health and awareness. It can be extended further by incorporating real-time data streams, advanced deep learning models, and integration with IoT sensors for continuous air quality assessment.

10.2 FUTURE SCOPE OF THE PROJECT:

The system can be extended to collect live air quality data from IoT sensors and government monitoring stations, enabling continuous AQI prediction and real-time alerts. Incorporating deep learning techniques such as LSTM (Long Short-Term Memory) networks can improve prediction accuracy by capturing temporal patterns in air quality data. Deploying the system on mobile applications and cloud platforms will make AQI predictions accessible to a wider audience and ensure scalability. Integration with GIS (Geographic Information Systems) can allow spatial visualization of AQI levels across different regions, helping policymakers and citizens understand pollution hotspots.

CHAPTER-11

11.1 REFERENCES

1. Yin, P.-Y. (2025). *A Review on PM2.5 Sources, Mass Prediction, and Association Analysis: Research Opportunities and Challenges*. Sustainability, 17(3), 1101. <https://doi.org/10.3390/su17031101>
2. Makhdoomi, A., Sarkhosh, M., & Ziaei, S. (2025). *PM2.5 Concentration Prediction Using Machine Learning Algorithms: An Approach to Virtual Monitoring Stations*. Scientific Reports, 15, Article 8076. <https://doi.org/10.1038/s41598-025-8076-3> ([doi.org in Bing](#))
3. International Journal of Innovative Research in Technology. (2024). *Predicting Air Quality Index (AQI) Using Machine Learning in Urban Environments*. IJIRT, Vol. 11, Issue 4, pp. 45–52.
4. Zhang, Y., et al. (2023). *Machine Learning-Based Quantification and Separation of Emissions and Meteorological Effects on PM2.5 in Greater Bangkok*. Atmospheric Environment, 287, 119–130.
5. Chen, L., & Li, J. (2022). *Deep Learning Approaches for Air Quality Forecasting: A Comparative Study*. Environmental Modelling & Software, 150, 105312.
6. Wang, T., & Zhao, H. (2021). *Hybrid Models Combining Statistical and Machine Learning Techniques for AQI Prediction*. Journal of Cleaner Production, 278, 123–135.
7. United States Environmental Protection Agency (EPA). (2020). *Technical Documentation for the Air Quality Index (AQI)*. EPA Report No. EPA-454/R-20-003.
8. Central Pollution Control Board (CPCB), India. (2019). *National Air Quality Index: Technical Guidelines and Implementation Report*. Ministry of Environment, Forest and Climate Change.
9. Li, X., & Guo, J. (2018). *Spatiotemporal Prediction of PM2.5 Concentrations Using LSTM Neural Networks*. Environmental Science & Technology, 52(20), 12344–12352.
10. Zheng, Y., et al. (2015). *Forecasting Fine-Grained Air Quality Based on Big Data*. Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2267–2276.